GSoC21 W4: LFortran, Backends and Bugs

Rohit Goswami · 2021-07-02 · gsoc21 · fortran · ramblings

> *Towards the mid-summer evaluation and redirecting efforts*

Background

Serialized update for the 2021 Google Summer of Code under the fortran-lang organization, mentored by Ondrej Certik.

Overview

This week was a bit of a sticky wicket. My unfamiliarity with the LLVM backend and its internals caught up with me just around the same time I ran into several fixture and rent related idiosyncrasies which led to yet another shift in my weekly meeting. Thanks to a timely reminder from my mentor Ondrej, I managed, nevertheless, to make some concrete progress working on the compilation of `dftatom` modules.

New Merge Requests

ASR Bug with Arrays (997)

Fixes a bug in the existing handling of arrays and `allocate` calls

Freshly Assigned Issues

Semantics: Implement TINY (388)

A classic intrinsic for error handling

Semantics: Array Initializer Expressions (389)

A bugfix related to the merge request

ODE1 Error (390)

Placeholder issue to be narrowed and chiseled into something actionable

ASR mismatch between `gfortran` modules and source (395)

Issue relating to the completeness and equivalence of `mod` formats

Additional Tasks

I'd like to make some time to continue exploring the LLVM, and continue familiarizing myself with formal language and automata theory.

Logistics

- I met with Ondrej on Friday this week
  - He was kind enough to make a standing meeting twice a week, Tuesdays and Fridays
  - This was done to ensure concrete progress is made beyond intangibles
- Discussed the possibility of extending the `lfortran mod` interface
  - Currently this has a copy of the ASR

    - In principle then, later modules are larger since they have relevant code
    - While `gfortran` does not bother with having a concrete representation in the `mod` file
- Discussed forming a functional subset of the `dftatom` code for now
    - This is related to the slight change in direction which is being considered
- Rather than stick to one feature / function from front end to back end, it was decided that a more breadth first approach would be better
    - Though I am still encouraged to dig into the backend, it is good to set up a fully working rational ASR pass first
- Eventually, backends aside, work will need to be undertaken for the runtime library as well

Misc Log

Some other things which came my way this week.

Linking Troubles

Sometime down the line of modules and the roadmap to `dftatom` described in the master issue here, I ran into an odd linker error. These seem to show up with both `miniconda` and `nix`, however, it is unclear to me at the moment if it warrants an issue until I check on a native ArchLinux machine. Another natural fix might be to actually bring in and compile `llvm`; then use the `lld` linker instead of the `bfd`.

```
lfortran mod types.mod --show-asr
Traceback (most recent call last):
  File "/build/glibc-2.32/csu/../sysdeps/x86_64/start.S", line 120, in _start()
  Binary file "/nix/store/hp8wcylqr14hrrpqap4wdrwzq092wfln-glibc-2.32-37/lib/libc.so.6", in
__libc_start_main()
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/bin/lfortran.cpp", line 1086, in ??
    asr = LFortran::mod_to_asr(al, arg_mod_file);
LFortranException: Unknown module file format
```

In any case, for now, switching to `micromamba` and using `direnv` just works.

Missing Symbols

This actually seems related to the more pertinent issue namely, that my `lapack` module didn't seem to contain relevant symbols at all:

```
lfortran -c types.f90 -o types.o
lfortran -c lapack.f90 -o lapack.o
lfortran -c constants.f90 -o constants.o
lfortran -c interpolation.f90 -o interpolation.o
Semantic error: The symbol 'dgesv' not found in the module 'lapack'
```

This is true actually:

```
cat lapack.mod | grep dgesv
```

Truncated Stacktraces

Also related, probably, was an issue I noticed while co-working with Ondrej, namely, that my stacktraces in `nix` were and are a good deal less informative than his, probably due to the default nix lack of debug symbols. This can be quickly reproduced with:

```
lfortran character_array.f90
BFD: DWARF error: could not find variable specification at offset 2722f
BFD: DWARF error: could not find variable specification at offset 2723e
BFD: DWARF error: could not find variable specification at offset 2736f
BFD: DWARF error: could not find variable specification at offset 2722f
BFD: DWARF error: could not find variable specification at offset 2723e
BFD: DWARF error: could not find variable specification at offset 2736f
BFD: DWARF error: could not find variable specification at offset 2722f
BFD: DWARF error: could not find variable specification at offset 2723e
BFD: DWARF error: could not find variable specification at offset 2736f
BFD: DWARF error: could not find variable specification at offset 2722f
BFD: DWARF error: could not find variable specification at offset 2723e
BFD: DWARF error: could not find variable specification at offset 2736f
BFD: DWARF error: could not find variable specification at offset 2722f
BFD: DWARF error: could not find variable specification at offset 2723e
BFD: DWARF error: could not find variable specification at offset 2736f
BFD: DWARF error: could not find variable specification at offset 2722f
BFD: DWARF error: could not find variable specification at offset 2723e
BFD: DWARF error: could not find variable specification at offset 2736f
BFD: DWARF error: could not find variable specification at offset 615b9
Traceback (most recent call last):
  File "/build/glibc-2.32/csu/../sysdeps/x86_64/start.S", line 120, in _start()
  Binary file "/nix/store/hp8wcylqr14hrrpqap4wdrwzq092wfln-glibc-2.32-37/lib/libc.so.6", in
__libc_start_main()
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/bin/lfortran.cpp", line 1250, in ??
    err = compile_to_object_file(arg_file, tmp_o, false,
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/bin/lfortran.cpp", line 601, in (anonymous
namespace)::compile_to_object_file(std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >
const&, bool, bool) [clone .isra.0]
    result = fe.get_asr2(input);
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/lfortran/codegen/evaluator.cpp", line 469, in
LFortran::FortranEvaluator::get_asr2(std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const&)
    asr = ast_to_asr(al, *ast, symbol_table);
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/lfortran/semantics/ast_to_asr.cpp", line 3091, in
LFortran::ast_to_asr(Allocator&, LFortran::AST::TranslationUnit_t&, LFortran::SymbolTable*)
    v.visit_TranslationUnit(ast);
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/lfortran/ast.h", line 3235, in
LFortran::SymbolTableVisitor::visit_TranslationUnit(LFortran::AST::TranslationUnit_t const&)
    case modType::Program: { v.visit_Program((const Program_t &)x); return; }
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/lfortran/ast.h", line 3555, in
LFortran::AST::BaseVisitor<LFortran::SymbolTableVisitor>::visit_program_unit(LFortran::AST::program_unit_t
const&)
    void visit_program_unit(const program_unit_t &b) { visit_program_unit_t(b, self()); }
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/lfortran/ast.h", line 3561, in
LFortran::AST::BaseVisitor<LFortran::SymbolTableVisitor>::visit_unit_decl2(LFortran::AST::unit_decl2_t const&)
    void visit_unit_decl2(const unit_decl2_t &b) { visit_unit_decl2_t(b, self()); }
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/lfortran/ast.h", line 3624, in
LFortran::AST::BaseVisitor<LFortran::SymbolTableVisitor>::visit_expr(LFortran::AST::expr_t const&)
    void visit_expr(const expr_t &b) { visit_expr_t(b, self()); }
  File "/users/home/rog32/Git/Github/Fortran/mylf/src/lfortran/ast.h", line 3369, in void
LFortran::AST::visit_expr_t<LFortran::SymbolTableVisitor>(LFortran::AST::expr_t const&,
LFortran::SymbolTableVisitor&)
    static void visit_expr_t(const expr_t &x, Visitor &v) {
  Binary file "/nix/store/hp8wcylqr14hrrpqap4wdrwzq092wfln-glibc-2.32-37/lib/libc.so.6", in killpg()
Segfault: Signal SIGSEGV (segmentation fault) received
```

With a rather simple test problem I was using for my `allocatable length` character feature.

```
! character_array.f90
program character_array
  implicit none

  character(15) :: something
  something = "sUpErWeiRdCaSe"

  call upcase(something)

  something = upcase_func(something)

  contains

subroutine upcase(s)
! Returns string 's' in uppercase
character(*), intent(in) :: s
character(len(s)) :: t
integer :: i, diff
t = s; diff = ichar('A')-ichar('a')
do i = 1, len(t)
    if (ichar(t(i:i)) >= ichar('a') .and. ichar(t(i:i)) <= ichar('z')) then
        ! if lowercase, make uppercase
        t(i:i) = char(ichar(t(i:i)) + diff)
    end if
end do
print*, "Subroutine"
print*, "Length of arg is ", len(s)
print*, "Converted " // s // " to " // t

end subroutine

function upcase_func(s) result(t)
! Returns string 's' in uppercase
character(*), intent(in) :: s
character(len(s)) :: t
integer :: i, diff
t = s; diff = ichar('A')-ichar('a')
do i = 1, len(t)
    if (ichar(t(i:i)) >= ichar('a') .and. ichar(t(i:i)) <= ichar('z')) then
        ! if lowercase, make uppercase
        t(i:i) = char(ichar(t(i:i)) + diff)
    end if
end do
! print*,  new_line('c')
print*, "Function"
print*, "Length of arg is ", len(s)
print*, "Converted " // s // " to " // t
end function

end program character_array
```

Gfortran Modules

One of the early goals was inter-operability with the `gfortran` module format. This is a `lisp` like syntax which changes occasionally.

```
! simple module
module b
implicit none
private
public g
contains
integer function g()
g = 5
end function
end module
```

The corresponding `asr` is given by:

```
(TranslationUnit (SymbolTable 1 {b: (Module (SymbolTable 2 {g: (Function (SymbolTable 3 {g: (Variable 3 g
ReturnVar () Default (Integer 4 []) Source Public Required)}) g [] [(= (Var 3 g) (ConstantInteger 5 (Integer 4
[])))] (Var 3 g) Source Public Implementation)}) b [] .false.)}) [])
```

For the currently supported `gfortran` module (v14) we get:

```
lfortran mod b14.mod --show-asr
(TranslationUnit (SymbolTable 1 {g: (Subroutine (SymbolTable 2 {}) g [] [] GFortranModule Public Interface)})
[])
```

Which is contains a lower amount of information. The differences between the module versions are not major:

```
b14.mod ⟶ bnz.mod

1:
GFORTRAN module version '14' created from b.f90
GFORTRAN module version '15' created from b.f90
(() () () () () () () () () () () () () () () () () () () () () ()
() () ())

14: GFORTRAN module version '14' created from b.f90

(2 'g' 'b' '' 1 ((PROCEDURE UNKNOWN-INTENT MODULE-PROC DECL UNKNOW
FUNCTION IMPLICIT_PURE) () (INTEGER 4 0 0 0 INTEGER ()) 0 0 () ()
0 0)
() 0 0)
)
```

More importantly however, the module versions correspond to being compressed and this needs to be eventually handled as well.

Conclusions

For the next week, I shall be finalizing work on the allocatable character lengths and moving on to some more semantics and intrinsics. Naturally as these are completed there will be more progress down the `dftatom` module listing. The main takeaway from this week for me was the retargeting of efforts. I do wish to continue struggling with the backend simply because I feel it is exciting and fun, but I shall endeavor to be more productive by applying a breadth first approach by contributing more concrete code every week. I have high hopes for being able to power through the intended project goals by the end of the program, and can always clean up / document afterwards.
